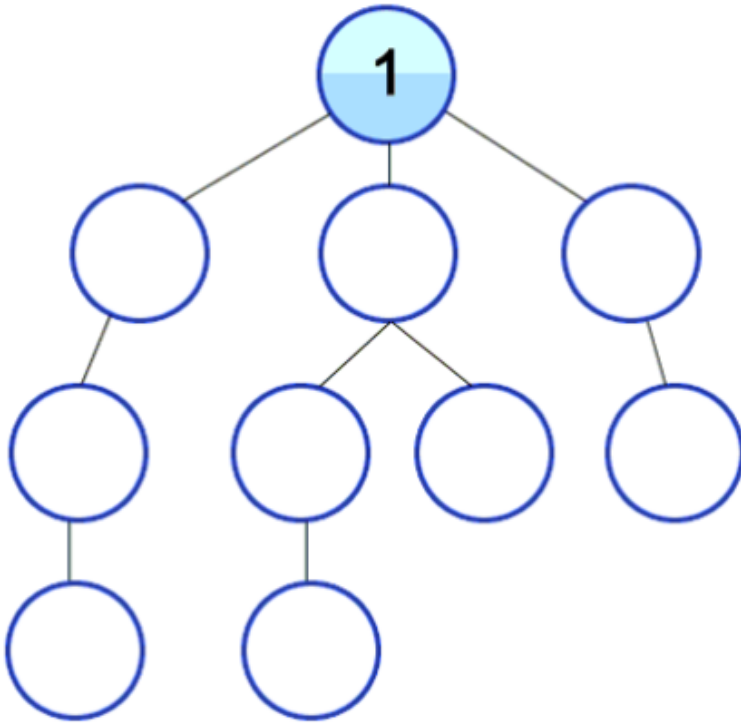


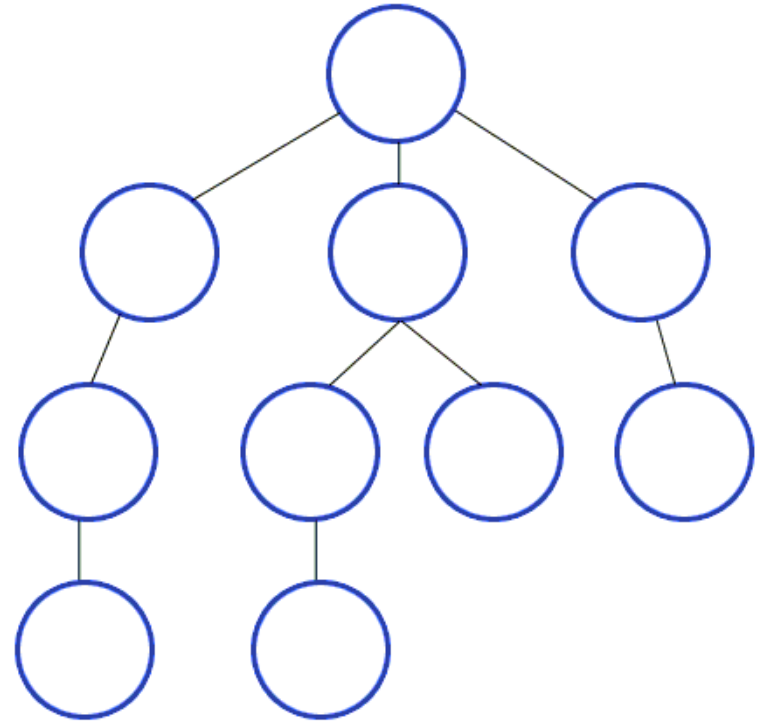


4.4 DFS คืออะไร

- The **Depth First Search** is fundamental search algorithm used to explore node and edges of a graph.
- It runs with a time $O(V+E)$
- Often used as a building block in other algorithms.



BFS



DFS



วัตถุประสงค์ & ขั้นตอนการทำงาน

- ตรวจสอบความลึกของกราฟ
- A DFS plunges depth first into a graph without regards for which edge it takes next until it cannot go any further at which point it backtracks and continues.
- DFS จะสำรวจกราฟแบบ **ลงลึกก่อน (Depth First)** โดยไม่สนใจมากนักว่าจะเลือกเส้นเชื่อม (edge) ใดเป็นเส้นถัดไป และจะสำรวจลงลึกต่อไปเรื่อย ๆ จนไม่สามารถไปต่อได้อีก จากนั้นจึง **ย้อนกลับ (backtrack)** ไปยังจุดก่อนหน้านี้ แล้วสำรวจเส้นทางอื่นต่อไป

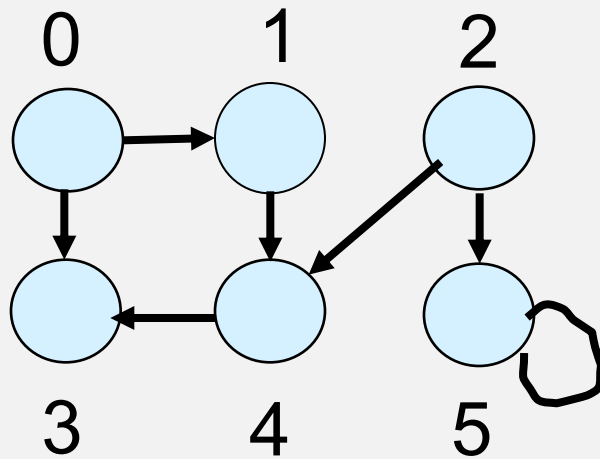


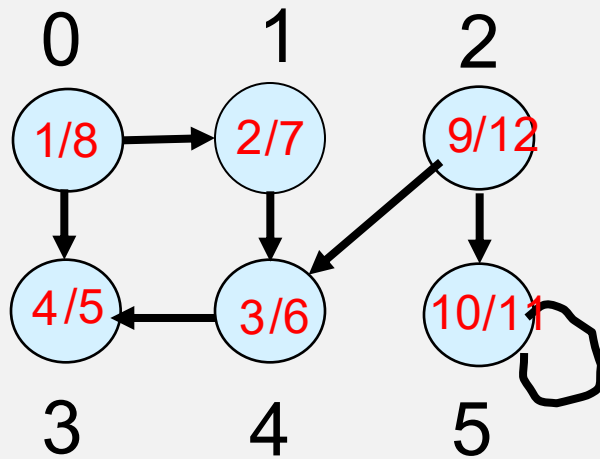
ขั้นตอน(ภาษาอังกฤษ)

1. Edges are explored out of the most recently discovered vertex v that still has unexplored edge leaving it.
2. When all of v 's edges have been explored, the search backtracks to explore edges leaving the vertex from which v has discovered. Until we have discovered all the vertices that are reachable from the original source vertex



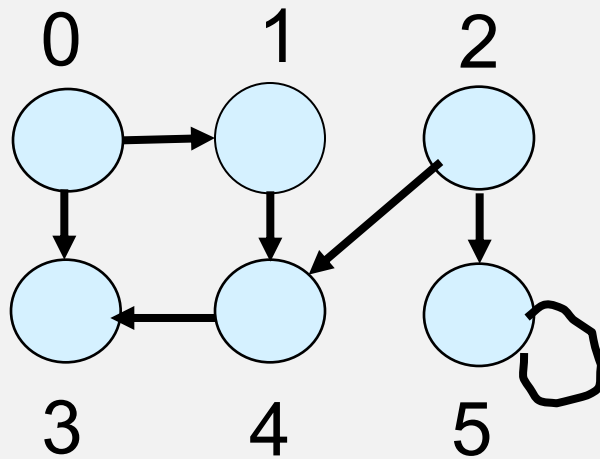
$S = 0$





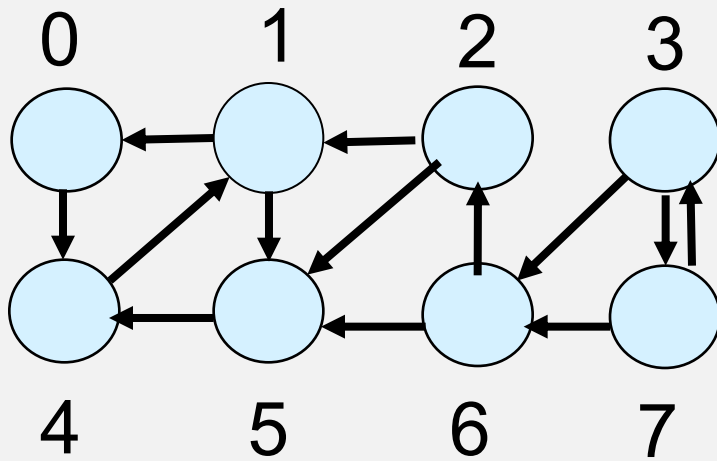


S= 4



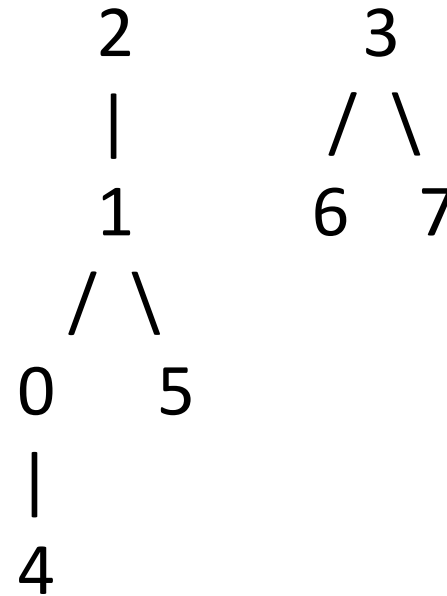
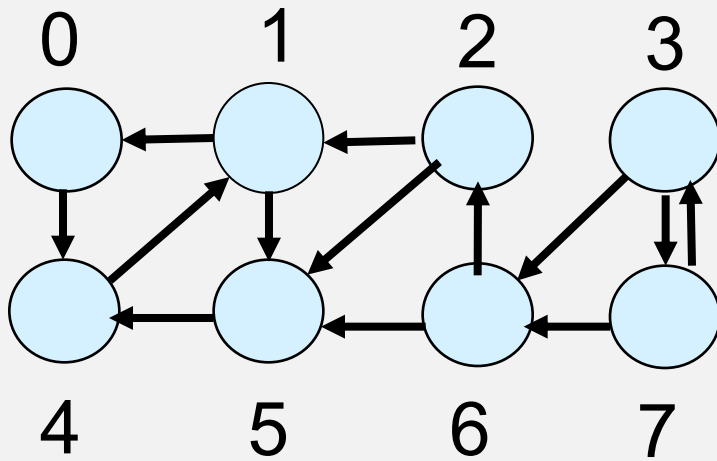


$S = 2$





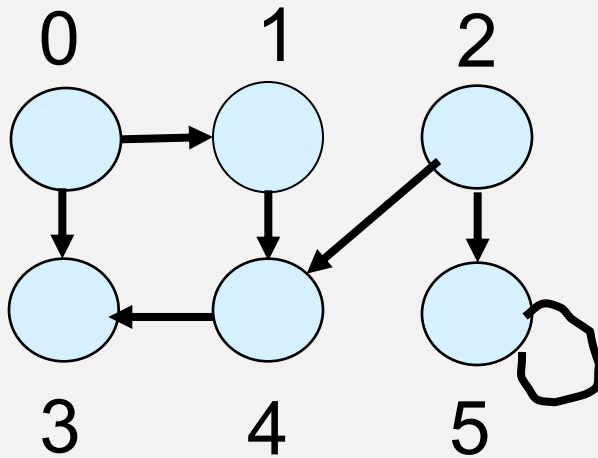
DFS trees





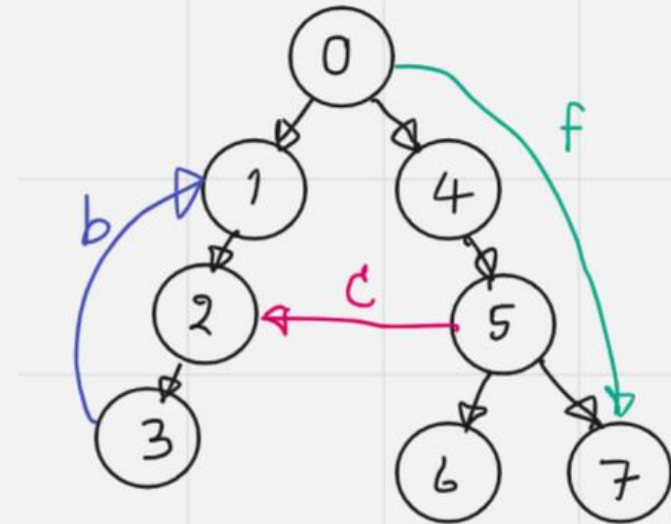
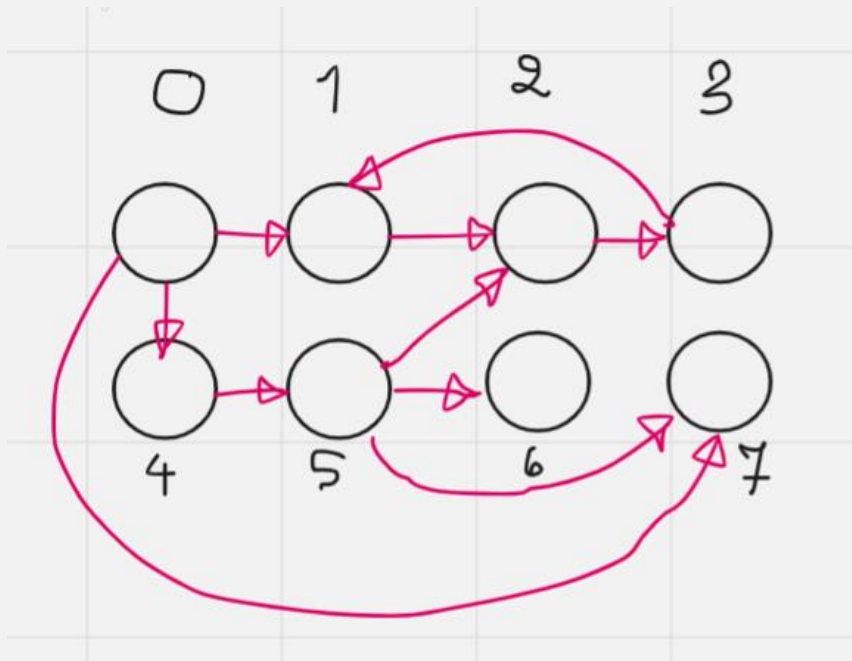
Edge Classification

$S = 0$





Edge Classification



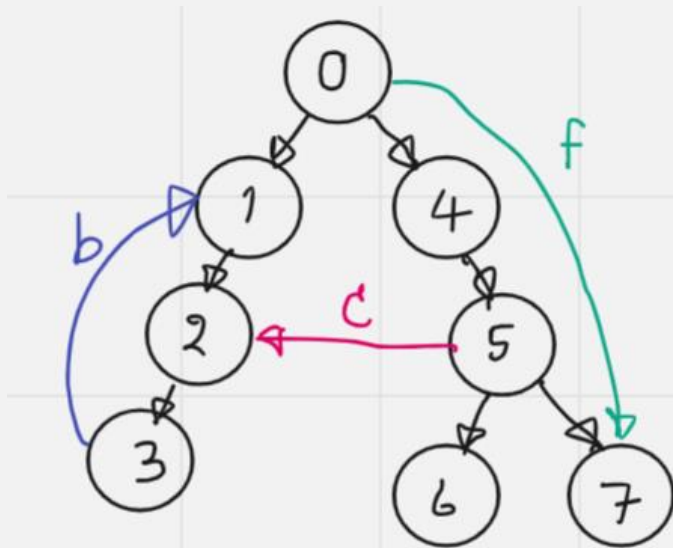


Edge Classification

1. Tree edges : are edges in the dfs forest G'' . Edge (u,v) is a tree edge if v was discovered by exploring edge (u,v) .

1. Back edge : are those edges (u,v) connecting a vertex u to an ancestor v in a dfs tree. Self-loop are considered to be back edges.

which point from a node to one of its ancestors



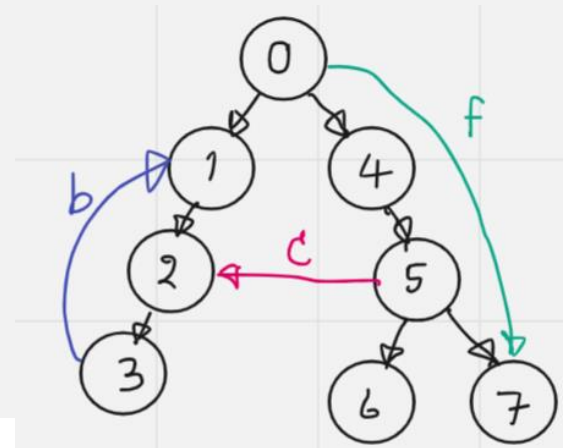


Classification of Edge

3. Forward edges : are those non tree edge (u,v) connecting a vertex u to a descendent v in a dfs tree.

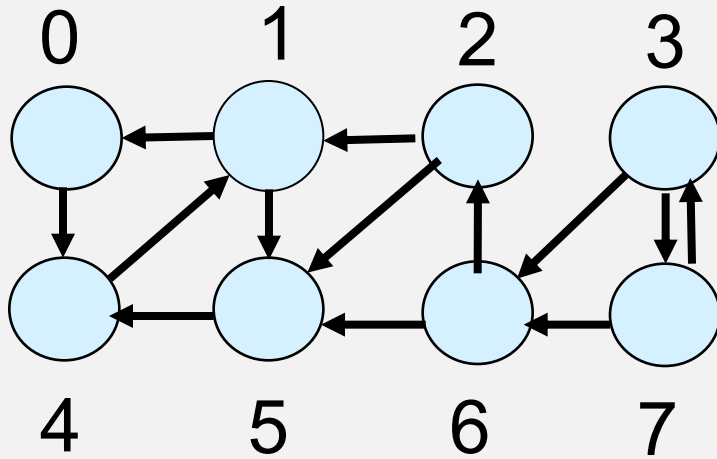
which point from a node of the tree to one of its descendants

4. Cross edge : are all other edges. They can go between vertices in the same depth-first search tree, as long as one vertex is not ancestor of the other, or they can go between vertices in difference dfs tree.

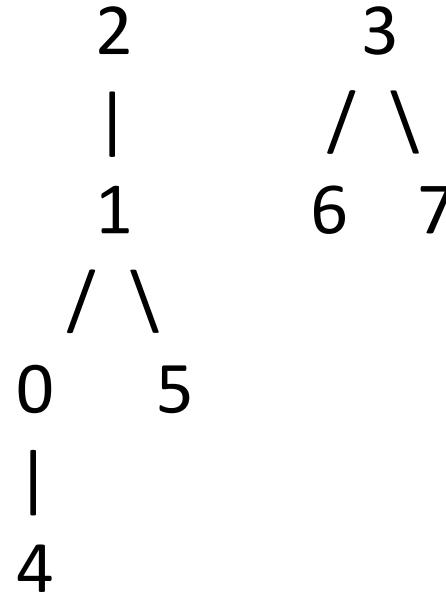




DFS trees



1. Tree edges
2. Back edges
3. Forward edges
4. Cross edges





4.4.2 Algorithm DFS (G)

DFS(G)

time=0;

for (u=0;u< 6; u++)

{ visited[u]=0

pred[u]=d[u]=f[u]=-1;

}

u=S //สมมติ s คือ โหนด 0

DFS_Visit(u)

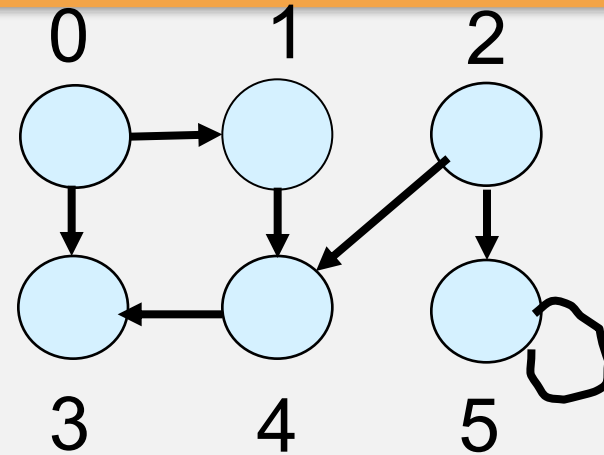
for (u=0;u<6;u++)

{ if (visited[u]=0)

{ DFS_Visit(u)

}

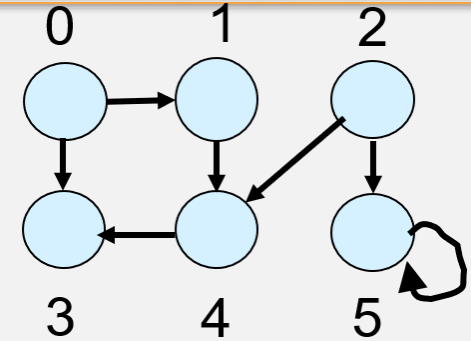
}



			visited	d	f	pred
0	1 3	0	0	-1	-1	-1
1	4	1	0	-1	-1	-1
2	4 5	2	0	-1	-1	-1
3		3	0	-1	-1	-1
4	3	4	0	-1	-1	-1
5	1	5	0	-1	-1	-1



```
// time=0 is global
DFS_Visit(u)
visited[u]=1
d[u]=++time;
for each v ∈ adj[u]
{
    if (visited[v] == 0)
    {
        pred[v] = u
        DFS_Visit(v)
    }
}
visited[u]=1
f[u]=++time;
```



			visited	d	f	pred
0		0	0	-1	-1	-1
1		1	0	-1	-1	-1
2		2	0	-1	-1	-1
3		3	0	-1	-1	-1
4		4	0	-1	-1	-1
5		5	0	-1	-1	-1

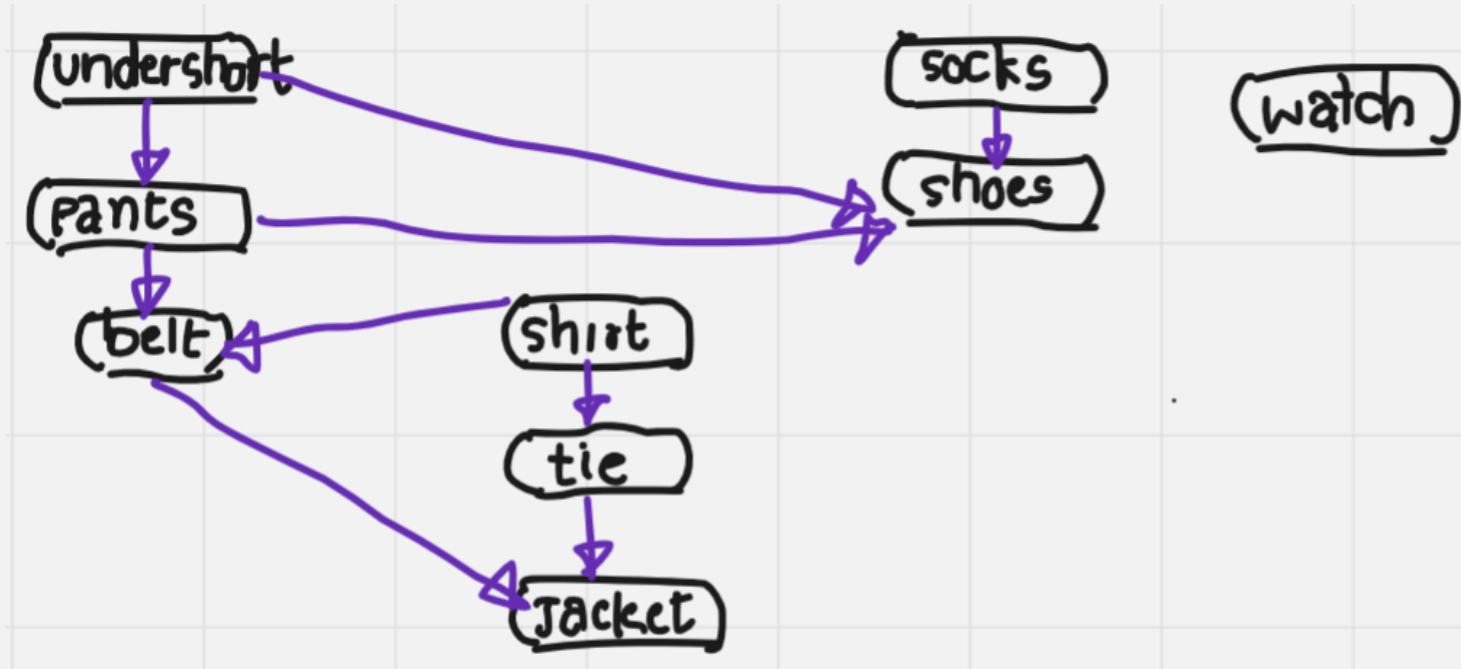


4.4.3 Topological sort

Topological sort of a DAG $G=(V,E)$ is a linear ordering of all its vertices such that if G contains an edge (u,v) , then u appears before v in the ordering. (If the graph is not acyclic, then no linear ordering is possible.)

Topological sort คือการนำเวอร์ทิซของกราฟกราฟหนึ่งมา

- เรียงเป็นเส้นตรงแบบมีลำดับ
- มีข้อกำหนดว่ากราฟนั้นจะต้องมีคุณสมบัติเป็น directed acyclic graph หรือ DAG (คือกราฟที่ไม่มีไซเคิล)





4.4.4 Algorithm Topological_sort(G)

Topological_sort(G)

1) call DFS(G) to computer finishing times $f[v]$ for each vertex v .

- ทำ DFS กราฟ



2) as each vertex is finished, insert it onto the front of a linked list

- เมื่อ DFS แล้ว ให้นำ vertex ที่มี finish time สูงสุดใส่ไว้ตำแหน่งแรกของ linklist

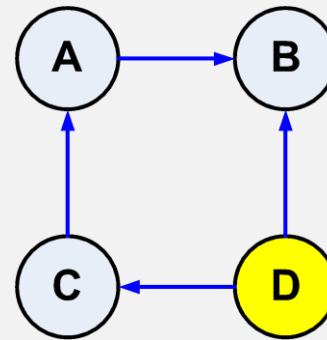
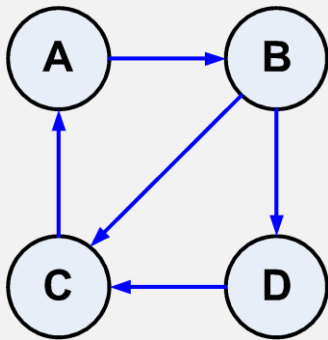
- finish time ที่มีค่ารองลงมาให้นำไปใส่ไว้ตำแหน่งที่ 2 ของ linklist และทำเช่นนี้โดยเรียงจากมากไปน้อย

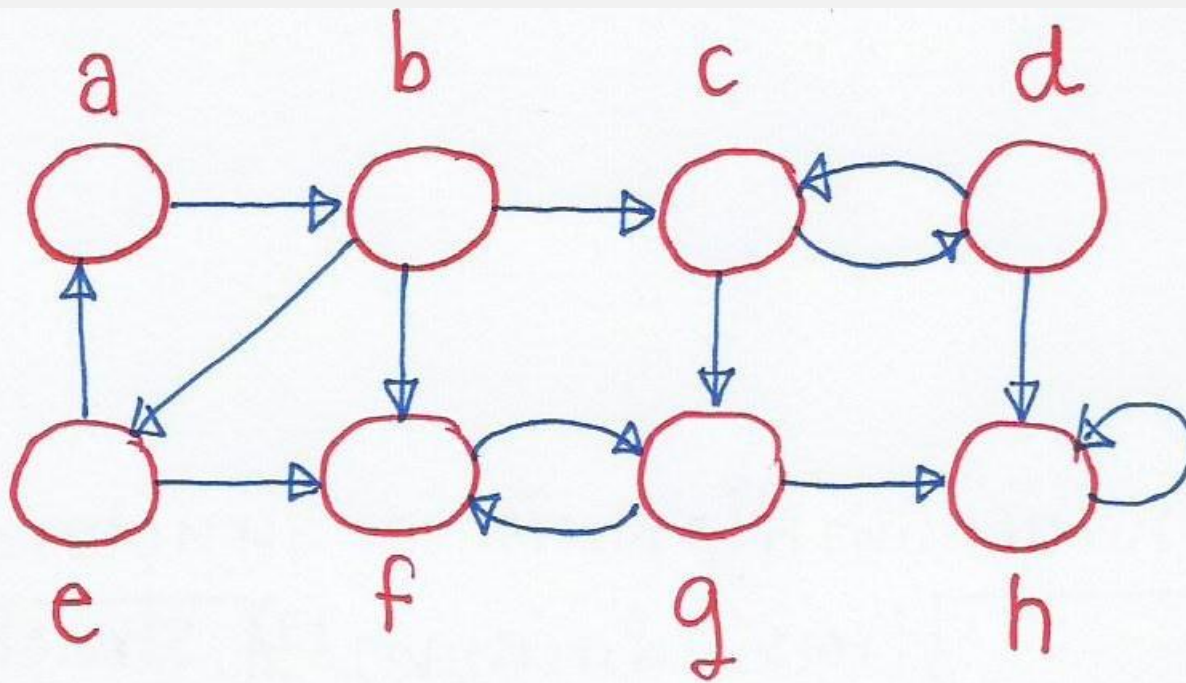
3) return the link list of vertices.

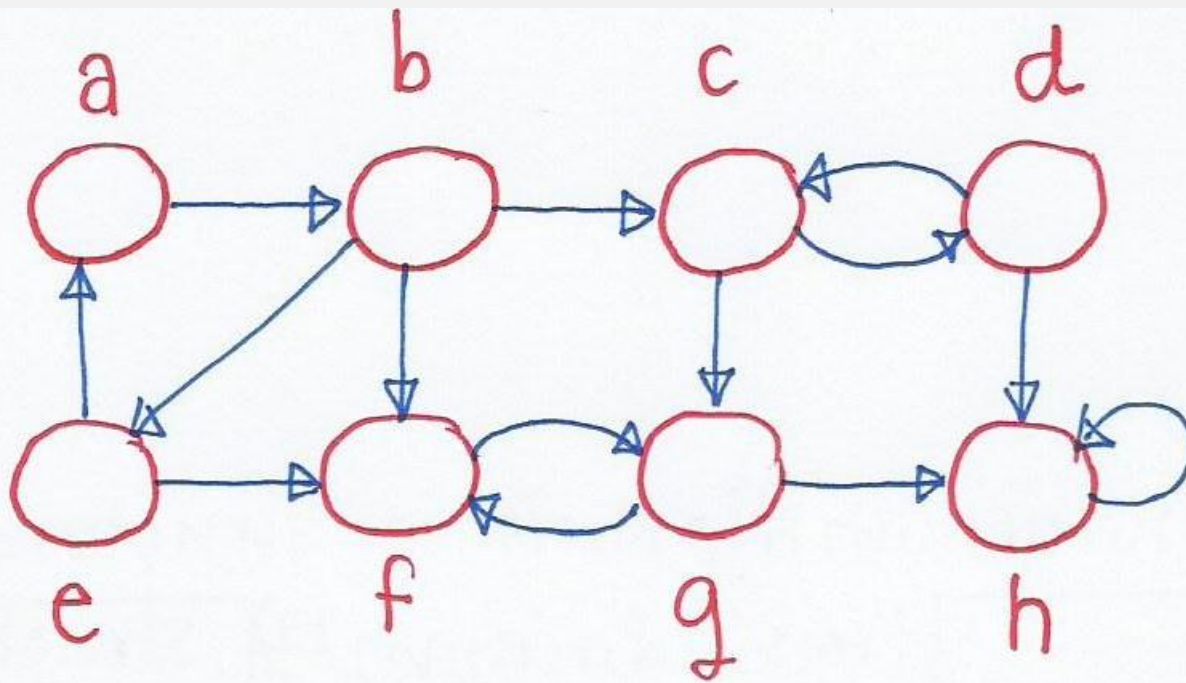


4.4.5 Strongly connected components

A digraph is ***strongly connected*** if every two vertices are reachable from each other. The ***strongly connected components*** of a graph are the equivalence classes of vertices under the “are ***mutually reachable***” relation.









4.4.6 Strongly-Connected-Component Algorithm

- 1) call DFS(G) to compute finishing times $f[u]$ for each vertex u
- 2) compute G^T (transpose graph)
- 3) call DFS(G^T), but the main loop of DFS, consider the vertices in order of decreasing $f[u]$ (as computed in line 1)
- 4) output the vertices of each tree



03603212 : Module4 – Graph

การบ้าน

1. เขียนโปรแกรม BFS

กำหนด input กราฟ และโหนดต้นทางที่จะทำการ BFS ผลลัพธ์คือระยะทางที่สั้นที่สุดจากโหนดต้นทางไปยัง ปลายทางทุกจุดที่สามารถ BFS ไปถึงได้

4 3

1 2

2 4

1 3

Start node : 1

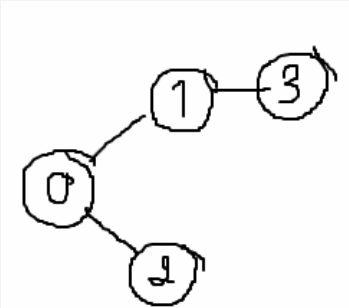
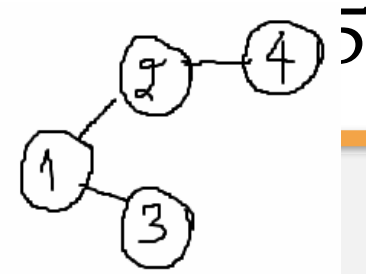
BFS

1, 0

2, 1

3, 1

4, 2





การบ้าน

2. เขียนโปรแกรม DFS

กำหนด input กราฟ และโหนดต้นทางที่จะทำการ BFS ผลลัพธ์คือระยะทางที่สั้นที่สุดจากโหนดต้นทางไปยัง ปลายทางทุกจุดที่สามารถ DFS ไปถึงได้

4 3

1 2

2 4

1 3

Start node : 1

Output

DFS

l d f pred

1 : 1 8 1

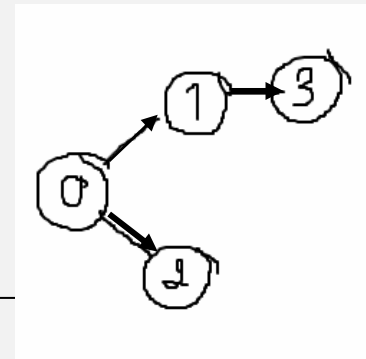
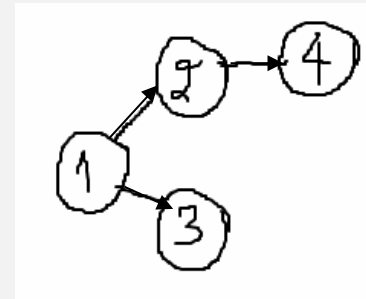
2 : 2 5 1

3 : 6 7 1

4 : 3 4 2

Enter end of Path : 4

1 2 4



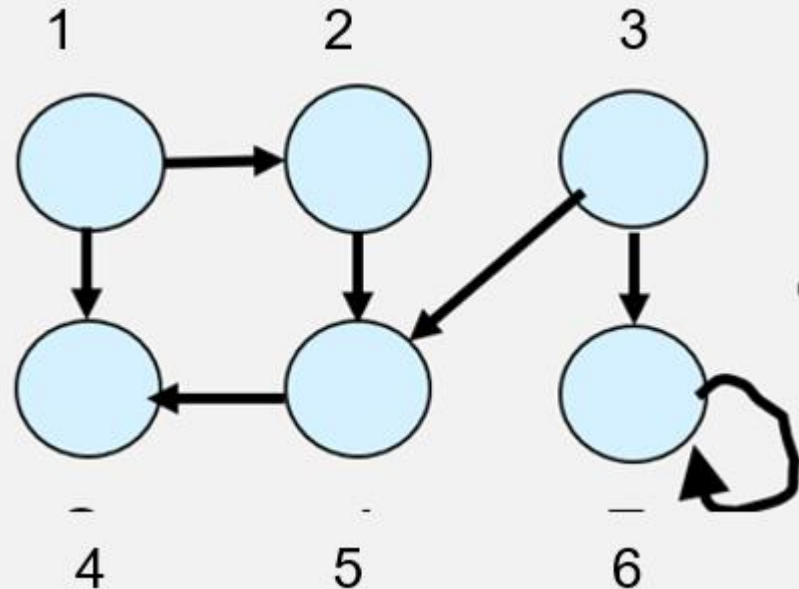


การบ้าน

3. ทำการ Traspose กราฟ

Input

6 7
1 2
1 4
2 5
5 4
3 6
6 6



Show

1: 2 4
2: 5
3: 5 6
4:
5: 4
6: 6

Transpose

1 :
2 : 1
3 :
4 : 1 5
5 : 2 3
6 : 3 6



การบ้าน

4. Topological sort เดี่ยว post ในที่มันจะคะ